

Day 3 — Learning Content

Analog I/O, Math & Data Types — full teaching script

1. Analog Inputs/Outputs (40 min)

Opening framing (say this to the class):

"Every input so far has been ON or OFF — a button, a switch, nothing in between. But a real temperature sensor doesn't jump straight from 0°C to 100°C — it moves through every value in between, continuously. That's an analog signal, and today is about how a PLC, which internally only understands numbers, deals with that."

1.1 — What the PLC actually receives

This is worth stating plainly, because it's the root of everything else today: **a PLC's analog input module has no idea what "°C" or "bar" means.** It only ever receives a raw number, typically somewhere in a fixed range like 0–1023 or 0–32767, representing the voltage or current the sensor produced. Two common industrial signal ranges are 4–20mA and 0–10V — mention these by name since students will encounter them on real datasheets.

Why 4–20mA and not 0–20mA — worth a full explanation, not a footnote:

Ask the class: "if a sensor's signal wire breaks, what should that look like to the PLC?" Let them guess before answering. With a 0–20mA range, a broken wire reads as 0mA — which looks identical to a valid "minimum reading" of 0. The PLC has no way to tell a real zero reading apart from a dead wire. With 4–20mA, a broken wire drops to 0mA, which is below the valid minimum of 4mA — so the PLC can detect that specific reading as an obvious fault rather than silently trusting a wrong value. This is a deliberate design choice in industrial sensors, not an arbitrary standard.

1.2 — The scaling formula, built up piece by piece

Don't present the full formula first — build it conceptually, then show the formula, then apply it. Start with this question: "if the raw value is exactly halfway between its minimum and maximum, what should the scaled value be?" (Answer: also halfway between its minimum and maximum.) This proportional relationship is the entire idea — the formula is just the general-case version of that.

The formula:

$$\text{Scaled Value} = ((\text{Raw Value} - \text{Raw Min}) / (\text{Raw Max} - \text{Raw Min})) \times (\text{Engineering Max} - \text{Engineering Min}) + \text{Engineering Min}$$

Read it aloud in plain language, piece by piece: "First, find how far into the raw range the current reading is, as a fraction between 0 and 1 — that's the (Raw Value - Raw Min) divided by (Raw Max - Raw Min) part. Then stretch that same fraction across the real-world range — multiply by the size of the engineering range. Then shift it up by the engineering minimum, in case that range doesn't start at zero."

Worked example 1 — zero-based range:

A sensor's raw output ranges from 0 to 1023, representing 0°C to 100°C. Raw reading = 512.

$$\text{Scaled} = ((512 - 0) / (1023 - 0)) \times (100 - 0) + 0 = 50.05^\circ\text{C}$$

Walk through each number as you substitute it: "512 minus 0 is 512. 1023 minus 0 is 1023. 512 divided by 1023 is about 0.5005 — just over halfway, since 512 is just over half of 1023. Multiply that by 100, the size of our temperature range, giving 50.05. Add the engineering minimum, which is 0, so the final answer stays 50.05°C."

Worked example 2 — a non-zero range, deliberately different from example 1:

A pressure sensor's raw output ranges from 200 to 800, representing 0 to 10 bar. Raw reading = 500.

$$\text{Scaled} = ((500 - 200) / (800 - 200)) \times (10 - 0) + 0 = 5.0 \text{ bar}$$

This example exists specifically to correct a misconception the first example can accidentally create — some students walk away assuming Raw Min is always 0. Point this out directly: "notice the raw range here doesn't start at 0 — it starts at 200. The formula still works exactly the same way, because we're always measuring distance from Raw Min, not from zero."

1.3 — Hands-on exercise

- 1 Connect a potentiometer (or simulated analog source) to an analog input
- 2 Read the raw value on screen — turn the knob slowly and watch it change continuously, not in fixed steps
- 3 Write the scaling formula into the program to convert the raw value into a 0–100°C range
- 4 Test at three points: fully anti-clockwise (should read close to 0°C), fully clockwise (should read close to 100°C), and roughly the middle (should read close to 50°C)
- 5 Extra practice: redo the scaling using worked example 2's non-zero range (200–800 raw → 0–10 bar) on the same physical input, to confirm the formula generalizes correctly

Instructor tip: test the two extreme ends of the potentiometer explicitly, not just the middle. Off-by-one range errors — using 1024 instead of 1023 as Raw Max, for instance — usually only show up as a small error right at the top end, and are easy to miss if you only ever test the middle of the range.

Checkpoint before moving on: give a student a new set of numbers on the spot — for example, "raw range 100 to 900, engineering range 20 to 120, raw reading of 500" — and have them calculate the scaled value on paper before checking it in software. (Answer: $(500-100)/(900-100) \times (120-20) + 20 = 70$.) If they can do this cold, without the two worked examples in front of them, the formula has genuinely landed rather than just being copied.

2. Arithmetic Instructions (25 min)

Opening framing:

"You've actually already used arithmetic — the scaling formula was subtraction, division, multiplication, and addition, all chained together. This section just introduces these as named, individual blocks you can use directly."

2.1 — The four blocks

- **ADD** — takes two or more values, outputs their sum

- **SUB** — takes two values, outputs the difference
- **MUL** — takes two or more values, outputs the product
- **DIV** — takes two values, outputs the quotient

2.2 — The DIV-by-zero hazard, worth real explanation

Ask the class: "what should happen if a program tries to divide a value by a variable that is currently zero?" Most students will guess correctly that this is undefined — but the important point for PLC work specifically is that this isn't just a math curiosity, it's a real operational hazard. If a divisor variable can legitimately be zero at some point during normal operation — for example, dividing a total by a count that starts at zero before anything has been counted yet — the program can fault, freeze, or produce an unpredictable result. This becomes directly relevant again on Day 4, when function blocks start performing calculations using variables that may not always have a safe value.

2.3 — Guided exercise

- 1 Take two analog readings (or one real reading plus one simulated value) and compute their average: ADD the two values, then DIV by 2
- 2 Display the result on screen and verify it updates live as either input changes
- 3 Extra practice: use SUB to compute the difference between the current reading and the previous scan's reading — this is the basic building block of detecting whether a value is currently rising or falling, which becomes useful again when discussing PID on Day 4

3. Comparison Instructions (20 min)

Opening framing:

"Arithmetic blocks produce numbers. But eventually, a number needs to become a decision — should the fan turn on or not? Comparison blocks are that bridge: they take a number and turn it into a simple ON/OFF result."

3.1 — The blocks

- **GT** — greater than
- **LT** — less than
- **EQ** — equal to
- **GE / LE** — greater-or-equal, less-or-equal (less commonly used, but worth naming)

3.2 — Why EQ is risky with decimal values — explain this properly, don't just warn against it

Ask the class: "if I calculate $0.1 + 0.2$ in the program, what should the result be?" Everyone will say 0.3. Explain that due to how decimal (REAL) numbers are actually stored in binary internally, the true stored result is sometimes something extremely close but not exactly 0.3 — like 0.30000000004. If a rung used EQ to check "is this value exactly equal to 0.3?", it would fail, even though the value is correct for all practical purposes. This is a genuinely common real-world bug, not a rare edge case — worth treating

seriously rather than as a minor footnote.

The practical rule to give students: for REAL (decimal) comparisons, use GT or LT (or GE/LE) instead of EQ wherever possible. EQ is safer reserved for whole-number (INT) comparisons, like checking whether a counter has reached exactly 10.

3.3 — Guided exercise

- 1 Using the scaled temperature from Section 1, build a GT comparison against a 50°C setpoint
- 2 Wire the comparison's output to a lamp representing a cooling fan
- 3 Sweep the potentiometer slowly and confirm the lamp switches on right at 50°C — not noticeably before or after
- 4 Extra practice: add a second LT comparison to also flag a "too cold" condition below 10°C, so the two comparisons together represent a normal operating band with an alert on either side

Checkpoint: ask a student to predict what the GT comparison's output will be at exactly 50.0°C — is it ON or OFF? (Correct answer: OFF — "greater than" is strictly greater than, not greater-or-equal, so exactly 50.0 does not satisfy GT 50. This is worth testing live, since it's a genuine ambiguity many students carry without realizing it.)

4. Data Types (20 min)

Opening framing:

"You've been using three different kinds of values all day without necessarily naming them — an ON/OFF lamp, a whole-number count, and a decimal temperature. This section just gives them proper names, and explains why the distinction actually matters."

4.1 — The three core types

- **BOOL** — ON/OFF, true/false only. Used for buttons, lamps, alarm flags. Takes the least memory of any type.
- **INT** — whole numbers, positive or negative (e.g., a part count). Cannot store any decimal portion at all.
- **REAL** — decimal numbers (e.g., 50.05°C). Required anywhere scaling or precise math is involved.

4.2 — A live demonstration worth running, not just describing

If any student, earlier today, tried storing a scaled value like 50.05 into a variable and saw it show up as 50 (rounded, decimal portion silently dropped), this is the moment to revisit that live on screen — deliberately store a decimal scaling result into an INT variable in front of the class and show the truncation happening. Seeing the data loss directly is far more convincing than being told about it in the abstract.

4.3 — Brief mention: WORD/DWORD

Worth a single sentence, not a full section: some real PLC hardware exposes raw binary/hex storage types called WORD or DWORD, distinct from INT even though they can look similar on screen. Students

likely won't use these directly this week, but may encounter the term when reading a real datasheet or existing program, so it's worth recognizing the name.

Checkpoint: ask which data type is correct for each of these three variables, and why: a motor's run/stop status, a part count, a scaled pressure reading. (Answers: BOOL, INT, REAL.) This is a fast check, but confirms the three core types are settled before the project pulls all of them together.

— BREAK — 15 min —

5. Day Project: Temperature Monitor (105 min)

Framing for the class before they start building:

"Every piece of this project is something you already built separately today — scaling, comparison, and a data type choice. The only new thing is combining all three into one working chain, where the output of one block becomes the input of the next."

5.1 — The specification to give students

- 1 Read a raw analog value from a simulated/real temperature sensor
- 2 Scale it to a 0–100°C range using the formula from Section 1
- 3 Compare the scaled value against a 50°C setpoint using GT — if above, turn ON a fan output
- 4 Compare the scaled value against a second, higher setpoint of 80°C — if above, turn ON a separate alarm output

5.2 — Why build order matters here

Tell students explicitly not to build all three blocks (scaling, fan comparison, alarm comparison) at once and then test everything together — this makes it very hard to know which block is at fault if something looks wrong. Instead:

- 1 Build the analog input and scaling block alone first. Display the scaled °C value on screen and confirm it looks correct across the full range of the potentiometer before adding anything else.
- 2 Add the fan comparison (GT 50°C) next. Test it in isolation — sweep the input and confirm the fan lamp switches at the right point.
- 3 Add the alarm comparison (GT 80°C) last. Test it independently, then test both comparisons together, confirming they behave correctly and independently of each other across the full sweep.

5.3 — The most common mistake, and how to guide a student to find it themselves

The single most frequent error in this project: comparing the raw, unscaled analog value directly against the 50°C or 80°C setpoint, instead of comparing the scaled value. Because raw values are often in a very different numeric range (like 0–1023) than the engineering values (0–100°C), this bug usually causes the fan to either never turn on, or turn on almost immediately at a very low knob position — both are strong tells.

If a pair hits this, don't name the bug directly. Instead ask: "which exact variable is the GT block's first input wired to — is that the raw value, or the value coming out of your scaling block?" This question alone resolves the issue in almost every case, and the discovery sticks far better than being told the answer outright.

5.4 — Extra challenge, for pairs who finish early

Add hysteresis to the fan control: the fan should turn ON at 50°C, but only turn back OFF once the temperature drops below 45°C — not switching OFF the instant it dips just under 50°C again. Ask students to think through why this matters before they build it: without hysteresis, a temperature hovering right at 50°C causes the fan to rapidly flicker on and off, which is hard on real motor hardware. This is also a natural bridge into Day 4's PID discussion, where smoother control is introduced properly.

End of day checkpoint: every student/pair demonstrates the fan and alarm outputs triggering at the correct temperatures, live, by sweeping the input across its full range — and can explain what would go wrong if the comparison blocks were accidentally wired to the raw value instead of the scaled value.